5

STATE-SPACE SEARCH

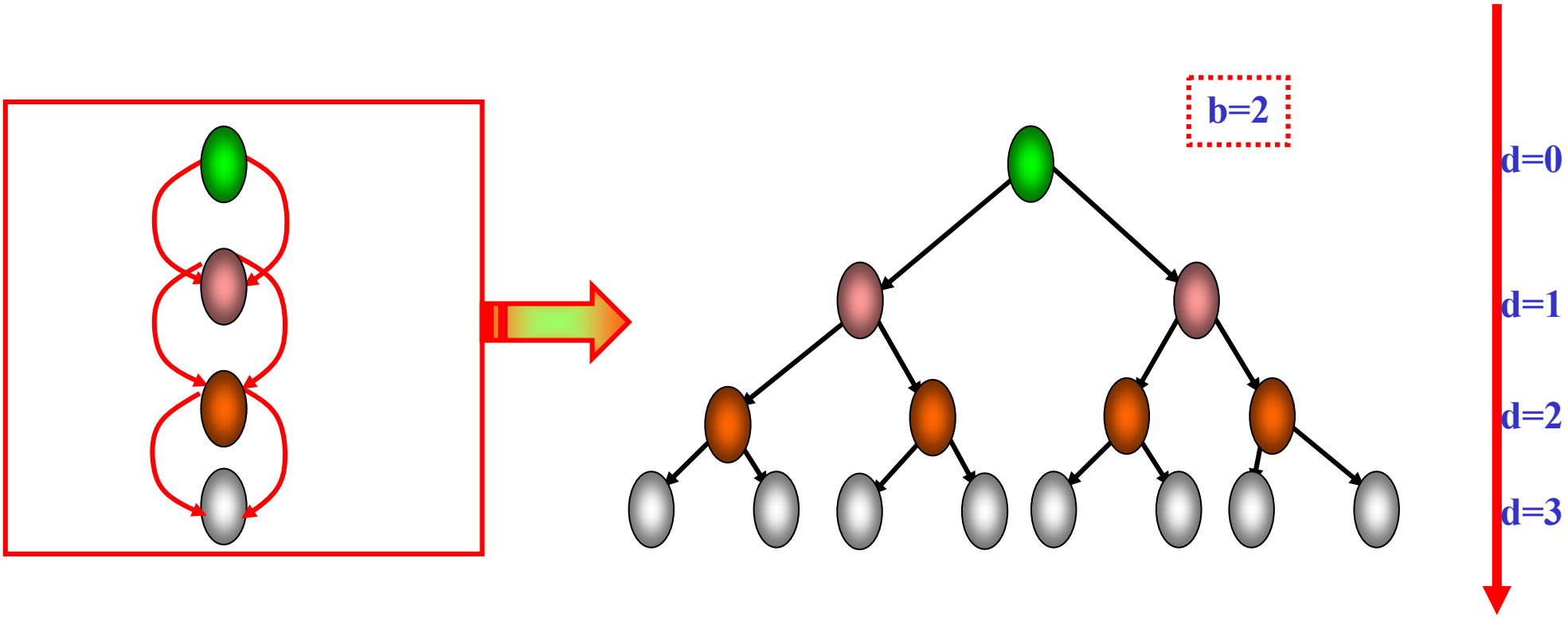
Optimal Search

A* and **Expanded List**

n.h.reyes@massey.ac.nz

Reference: MIT Open Courseware

States vs. Paths

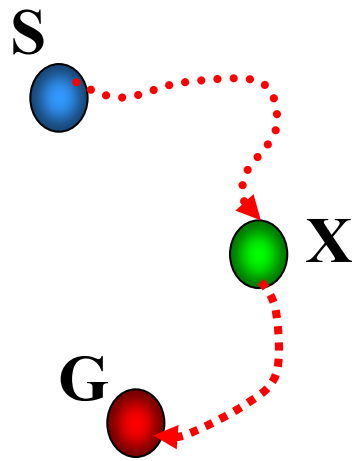


We have ignored the issue of revisiting states in our discussion of Uniform-Cost Search and A*. We indicated that we could not use the Visited List and still preserve optimality, but can we use something else that will keep the worst-case cost of a search proportional to the number of states in a graph rather than to the number of non-looping paths?

Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the **shortest path** from **S** to **X** and the **shortest path** from **X** to **G**. This is the "dynamic programming optimality principle".

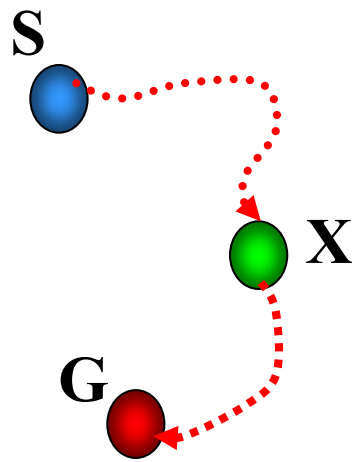


Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the **single best path** from **S** to any state **X**;
If we find a new path to a state already in **Q**, **discard the longer one**.



Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the single best path from **S** to any state **X**; If we find a new path to a state already in **Q**, discard the longer one.

Note that the first time **UC** pulls a search node off of **Q** whose state is **X**, this path is the shortest path from **S** to **X**. This follows from the fact that **UC** expands nodes in order of actual path length.

Uniform Cost Search

Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the single best path from **S** to any state **X**; If we find a new path to a state already in **Q**, discard the longer one.

Note that the first time **UC** pulls a search node off of **Q** whose state is **X**, this path is the shortest path from **S** to **X**. This follows from the fact that **UC** expands nodes in order of actual path length.

So, once we expand one path to state **X**, we don't need to consider (extend) any other paths to **X**. We can keep a list of these states, call it **Expanded List**. If the state of the search node we pull off of **Q** is in the Expanded List, we discard the node. When we use the Expanded List this way, we call it "**strict**"

Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the single best path from **S** to any state **X**; If we find a new path to a state already in **Q**, discard the longer one.

Note that the first time **UC** pulls a search node off of **Q** whose state is **X**, this path is the shortest path from **S** to **X**. This follows from the fact that **UC** expands nodes in order of actual path length.

So, once we expand one path to state **X**, we don't need to consider (extend) any other paths to **X**. We can keep a list of these states, call it **Expanded List**. If the state of the search node we pull off of **Q** is in the Expanded List, we discard the node. When we use the Expanded List this way, we call it "**strict**"

Note that **UC** without this is still correct, but inefficient for searching graphs.

SEARCH

Simple Optimal Search Algorithm: Uniform Cost

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry.
2. If Q is empty, fail. Else, pick search node N with **lowest path cost** from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. -
6. Find all the children of state(N) and create all the one-step extensions of N to each descendant.
7. Add all the extended paths to **front** of Q. (our convention in class)
8. Go to Step 2.

back

SEARCH

Simple Optimal Search Algorithm: Uniform Cost + Strict Expanded List

A Search Node is a path from some state X to the start state

e.g. (X B A S)

The state of a search node is the most recent state of the path

e.g. X

Let Q be a list of search nodes

e.g. (X B A S) (C B A S) …)

Let S be the start state.

Take note that we need to add some precautionary measures when adding extended paths to Q.



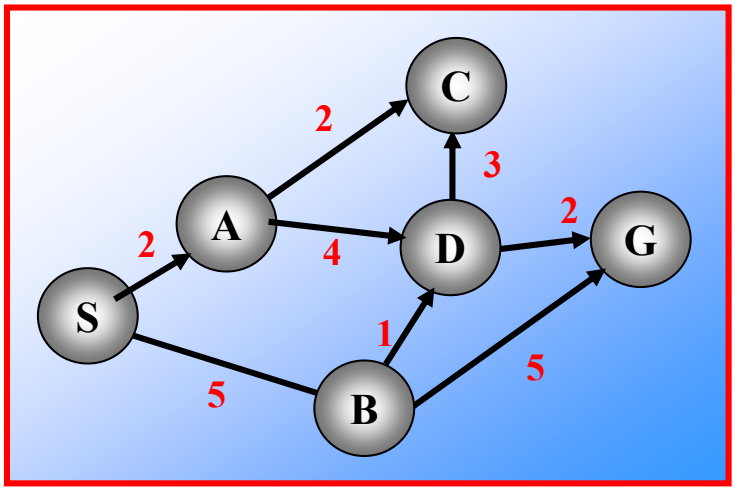
1. Initialise Q with search node (S) as only entry; **set Expanded = ( )**
2. If Q is empty, fail. Else, pick some search node N with **lowest path length** from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.

In effect, discard that search node N

5. **If state (N) in Expanded, go to Step 2; otherwise, add state (N) to Expanded List.**
6. Find all the children of state(N) **(Not in Expanded)** and create all the one-step extensions of N to each descendant.
7. **Add all the extended paths to front of Q. If descendant state already in Q, keep only shorter path to state in Q.**
8. Go to Step 2.

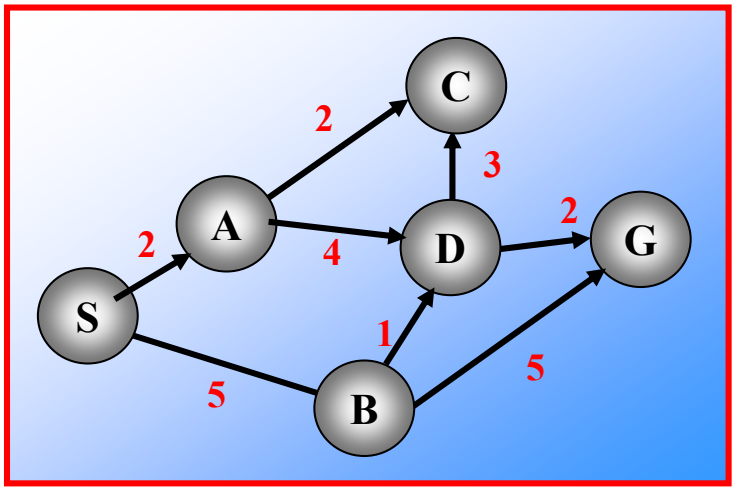
Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	(2 AS)(5 BS)	S



Uniform Cost (with Strict Expanded List)

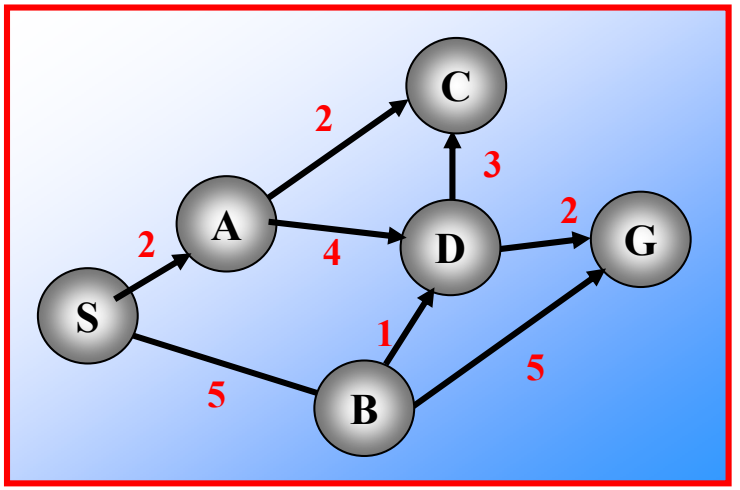
Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)	S, A



SEARCH

Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)(5 BS)	S, A, C

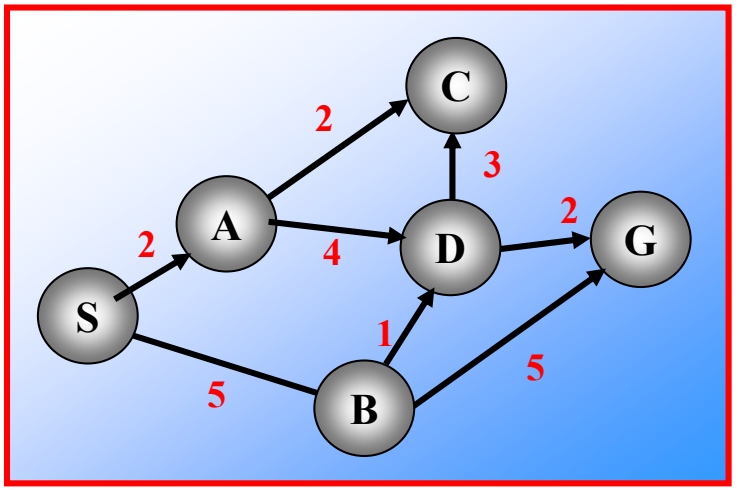


back

SEARCH

Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)~~(5 BS)~~	S, A, C
4	(5 BS)	~~(6 DBS)~~(10 GBS)~~(6 DAS)~~	S, A, C, B



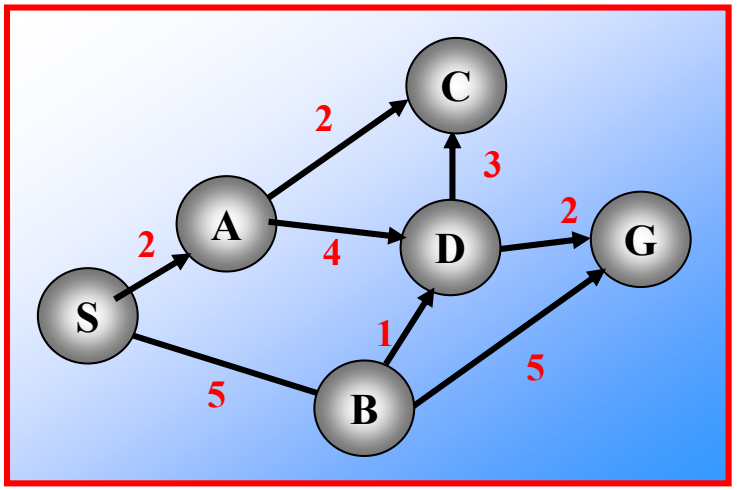
Removed because D is already in Q (our convention: take element at the front of the Q if a path leading to the same state is in Q).

back

SEARCH

Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)~~(5 BS)~~	S, A, C
4	(5 BS)	~~(6 DBS)~~(10 GBS)~~(6 DAS)~~	S, A, C, B
5	(6 DAS)	(8 GDAS)~~(10 GBS)~~	S, A, C, B, D

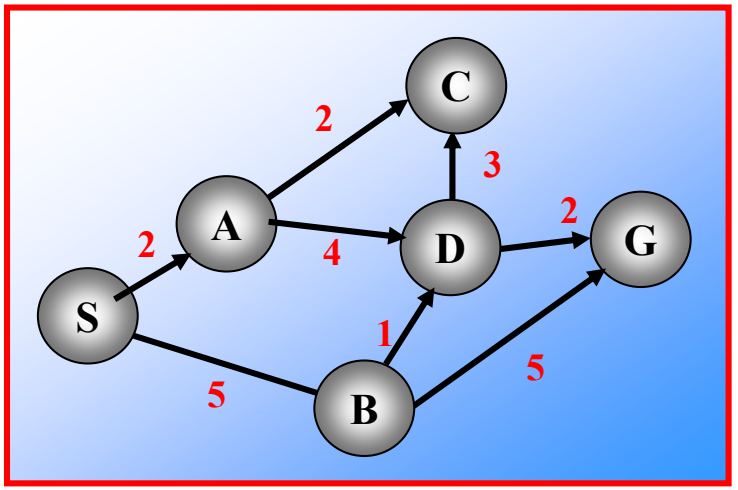


Removed because D is already in Q (our convention: take element at the front of the Q if a path leading to the same state is in Q).

Removed because there's a new shorter path to G

Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)~~(5 BS)~~	S, A, C
4	(5 BS)	~~(6 DBS)~~(10 GBS)~~(6 DAS)~~	S, A, C, B
5	(6 DAS)	~~(8 GDAS)~~~~(10 GBS)~~	S, A, C, B, D
6	(8 GDAS)		



Removed because D is already in Q (our convention: take element at the front of the Q if a path leading to the same state is in Q).

Removed because there's a new shorter path to G

Sequence of State Expansions: S – A – C – B – D - G

Sequence of state expansions = S–A–C–B–D-G

Path = S-A-D-G

Path length = 8

Max Q Length = 3

Number of state expansions = 6

UC with the Expanded List vs. UC without the Expanded List

	UC with the Expanded List	UC without the Expanded List
Sequence of state expansions	S–A–C–B–D-G	S– A – C – B – D – D – G
Path	S-A-D-G	S-A-D-G
Path Length	8	8
Max Q Length	3	5
Number of state expansions	6	7

back

A* (without Expanded List)

Let **g(N)** be the path cost of **N**, where **N** is a search tree node, i.e. a partial path.

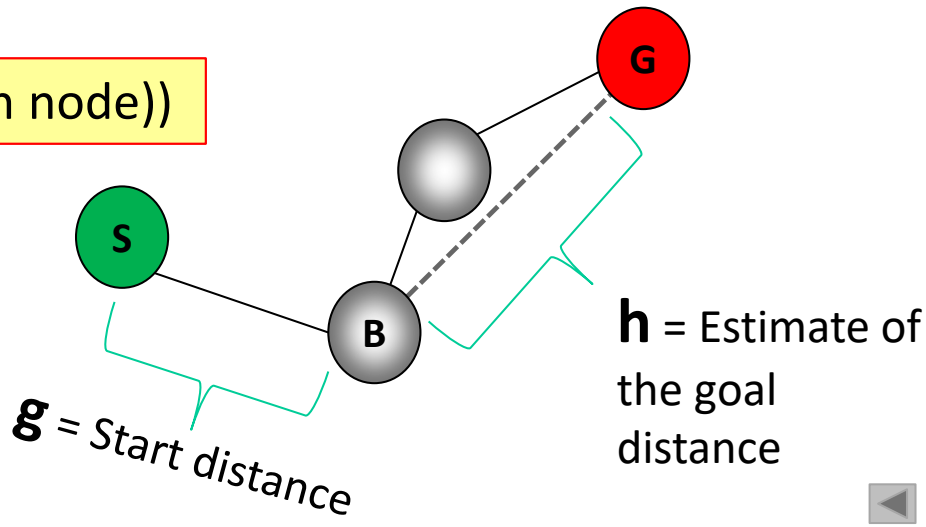
Let **h(N)** be h(state(N)), the heuristic estimate of the remaining path length to the goal from state(N).

Let **f(N) = g(N) + h(state(N))** be the total estimated path cost of a node, i.e. the estimate of a path to a goal that starts with the path given by N.

A* picks the node with **lowest f value** to expand.

f(search node)= g(search node) + h(state(search node))

e.g. f(BS) = g(S→B) + h(state(BS))



back

A* (without Expanded List)

Let g(N) be the path cost of N, where N is a search tree node, i.e. a partial path.

Let h(N) be h(state(N)), the heuristic estimate of the remaining path length to the goal from state(N).

Let f(N) = g(N) + h(state(N)) be the total estimated path cost of a node, i.e. the estimate of a path to a goal that starts with the path given by N.

A* picks the node with lowest f value to expand.

A* (**without Expanded List**) and with **admissible heuristic** is **guaranteed** to find optimal paths – those with smallest path cost.

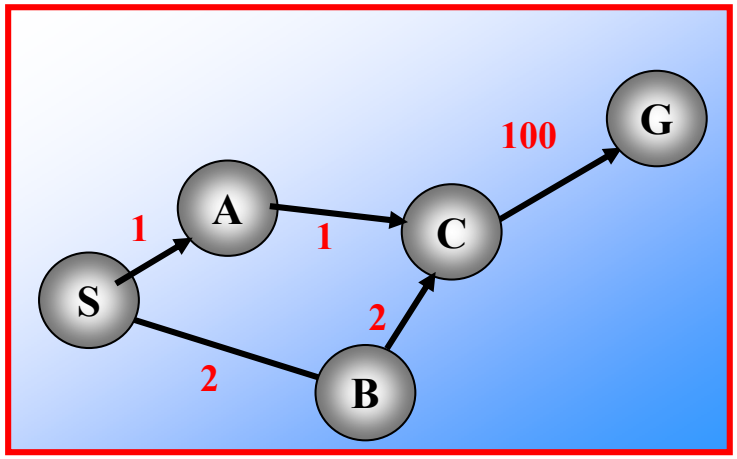
A* and the Strict Expanded List

The **Strict Expanded List** (also known as a **Closed List**) is commonly used in implementations of A* but, to guarantee finding optimal paths, this implementation requires a ***stronger condition*** for a heuristic than simply being an underestimate.

A* and the Strict Expanded List

The Strict Expanded List (also known as a Closed List) is commonly used in implementations of A* but, to guarantee finding optimal paths, this implementation requires a ***stronger condition*** for a heuristic than simply being an underestimate.

Here's a counterexample: The heuristic values listed below are all underestimates but A* using an Expanded List will not find the optimal path.



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

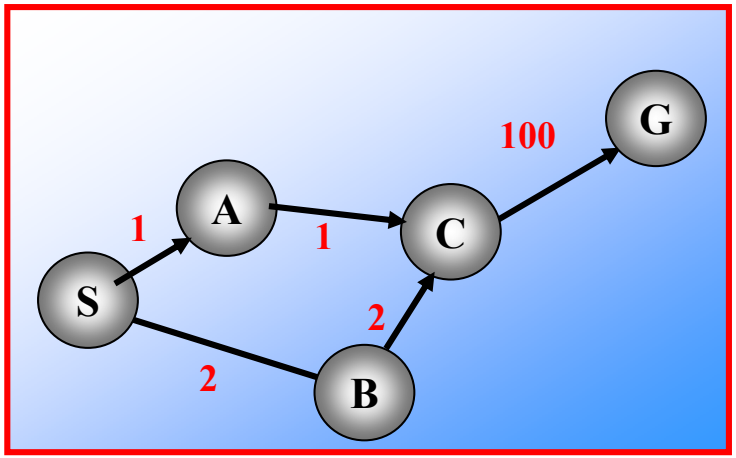
Step	Dequeued	Q	Expanded List
0		(90 S)	

f= g + h

f(S) = g(S) + h(state(S))

f(S) = 0 + h(S)

= 0 + 90 = 90



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions to **front** of Q.

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

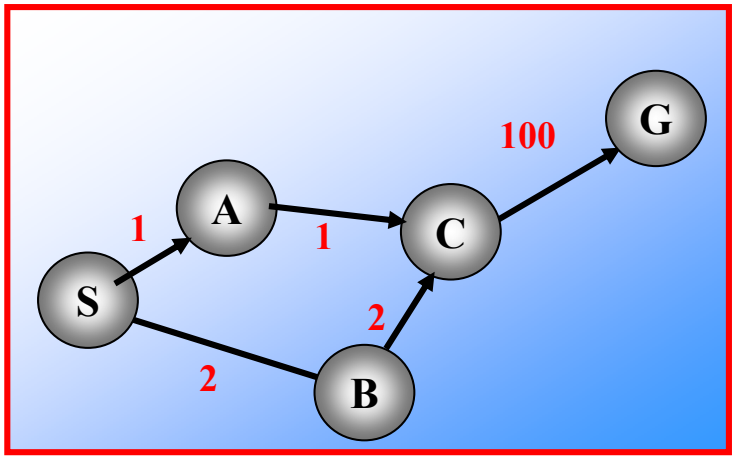
Step	Dequeued	Q	Expanded List
0		(90 S)	
1	(90 S)	(101 AS)(3 BS)	S

f= g + h

$$f(AS) = g(S \rightarrow A) + h(state(AS))$$

$$f(AS) = 1 + h(A)$$

$$= 1 + 100 = 101$$



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions to **front** of Q.

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

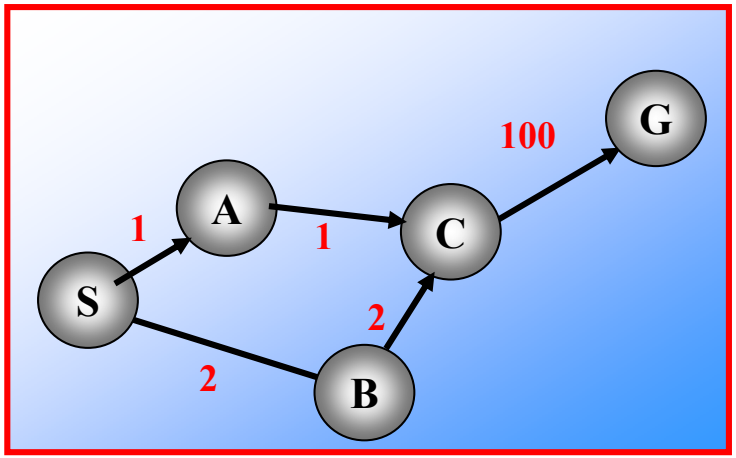
Step	Dequeued	Q	Expanded List
0		(90 S)	
1	(90 S)	(101 AS)(3 BS)	S

f= g + h

$$f(BS) = g(S \rightarrow B) + h(state(BS))$$

$$f(BS) = 2 + h(B)$$

$$= 2 + 1 = 3$$



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

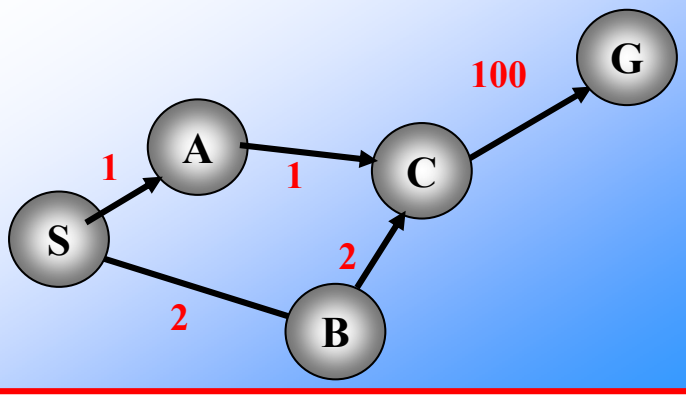
- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions to **front** of Q.

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

Step	Dequeued	Q	Expanded List
0		(90 S)	
1	(90 S)	(101 AS)(3 BS)	S
2	(3 BS)	(94 CBS) (101 AS)	B, S
3	(94 CBS)	(104 GCBS)(101 AS)	C, B, S
4	(101 AS)	(104 GCBS)	A, C, B, S
5	(104 GCBS)		G, A, C, B, S



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

back

Consistency

To enable implementing A* using the strict Expanded List, **H** needs to satisfy the following **consistency** (also known as **monotonicity**) conditions

1. $h(S_i) = 0$, if n_i is a goal
2. $h(S_i) - h(S_j) <= c(S_i, S_j)$, if n_j a child of n_i

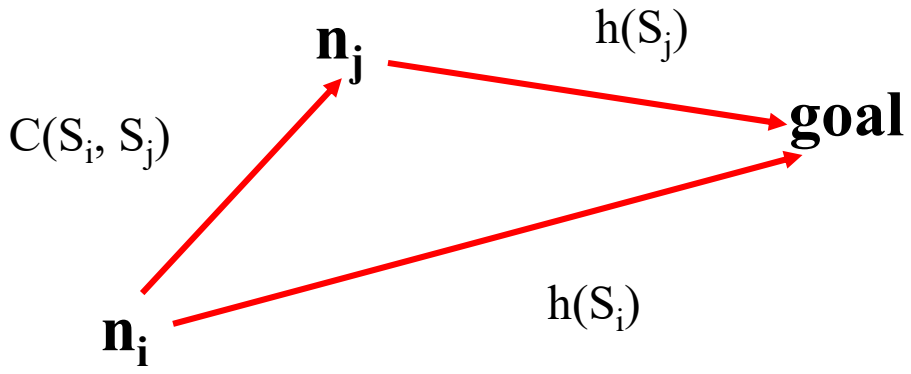
Consistency

To enable implementing A* using the strict Expanded List, H needs to satisfy the following **consistency** (also known as **monotonicity**) conditions

1. $h(S_i) = 0$, if n_i is a goal

2. $h(S_i) - h(S_j) <= c(S_i, S_j)$, if n_j a child of n_i

That is, the heuristic cost in moving from one entry to the next cannot decrease by more than the arc cost between the states. This is a kind of ***triangle inequality*** This condition is a highly desirable property of a heuristic function and often simply assumed (more on this later).



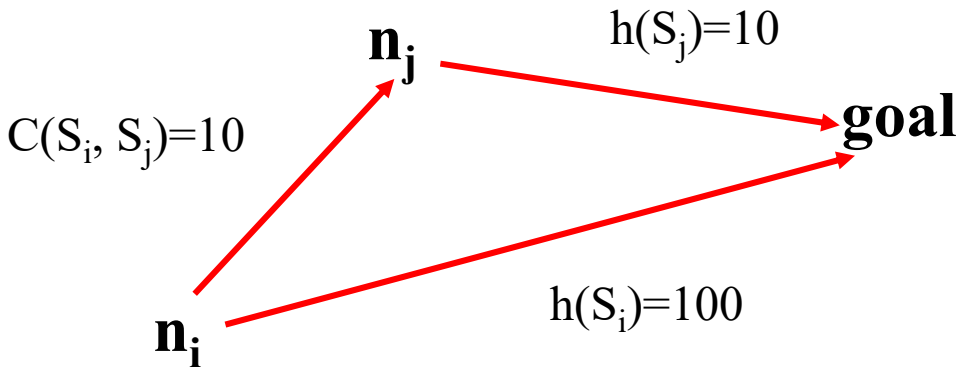
Consistency Violation!

A simple example of violation of consistency:

1. $h(S_i) = 0$, if n_i is a goal
2. $h(S_i) - h(S_j) <= c(S_i, S_j)$, if n_j a child of n_i

In the example, 100-10 > 10

If you believe that the **goal** is **100** units from n_i, then moving 10 units to n_j should not bring you to a distance of **10** units from the goal.



?

$100-10 \leq 10$

This evaluates to false. Violates consistency requirement!

A* (without Expanded List)

Let **g(N)** be the path cost of n, where n is a search tree node, i.e. a partial path.

Let **h(N)** be **h(state(N))**, the heuristic estimate of the remaining path length to the goal from state(N).

Let **f(N) = g(N) + h(state(N))** be the total estimated path cost of a node, i.e. the estimate of a path to a goal that starts with the path given by N.

A* picks the node with **lowest f value** to expand.

A* (without Expanded List) and **with admissible heuristic** is **guaranteed** to find optimal paths – those with smallest path cost.

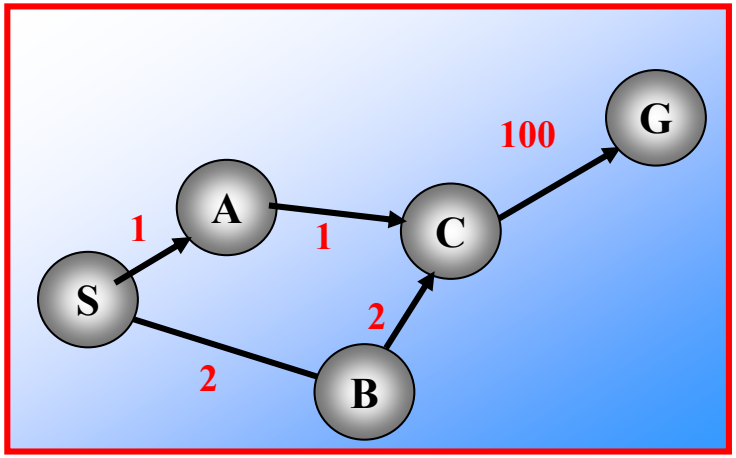
This is true even if heuristic is **NOT** consistent.

back

A* (without the Strict Expanded List)

Step		Q
0		(90 S)
1	(90 S)	(3 BS)(101 AS)
2	(3 BS)	(94 CBS) (101 AS)
3	(94 CBS)	(101 AS)(104 GCBS)
4	(101 AS)	(92 CAS)(104 GCBS)
5	(92 CAS)	(102 GCAS)(104 GCBS)
6	(102 GCAS)	

Note that the heuristic is admissible but **NOT** consistent.



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

inconsistent!

Underlined paths are chosen for extension.

This heuristic is not consistent but it is an **underestimate** and that is all that is needed for **A* without an Expanded List** to guarantee optimality.

A* (with Strict Expanded List)

Just like Uniform Cost Search.

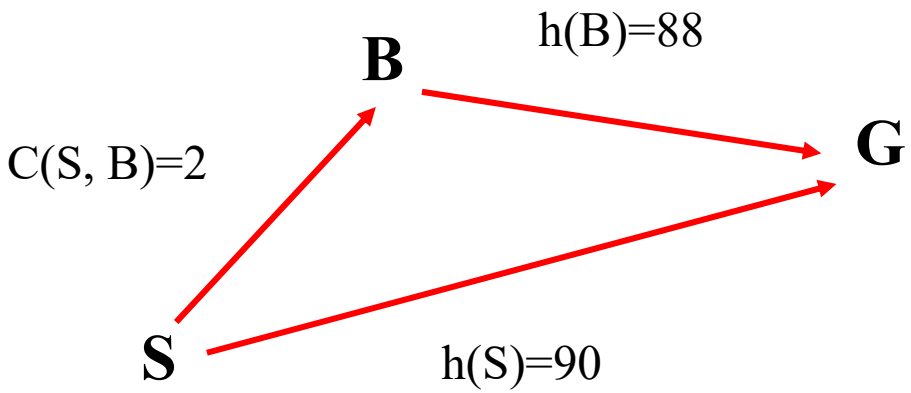
When a node **N** is expanded, if state **(N)** is in Expanded List, discard **N**, else add state **(N)** to Expanded List.

If some node in **Q** has the same state as some descendant of **N**, keep only node with smaller **f**, which will also corresponds to smaller **g**.

For **A* (with Strict Expanded List)** to be guaranteed to find the optimal path, the heuristic must be **consistent**.

A* (with Strict Expanded List)

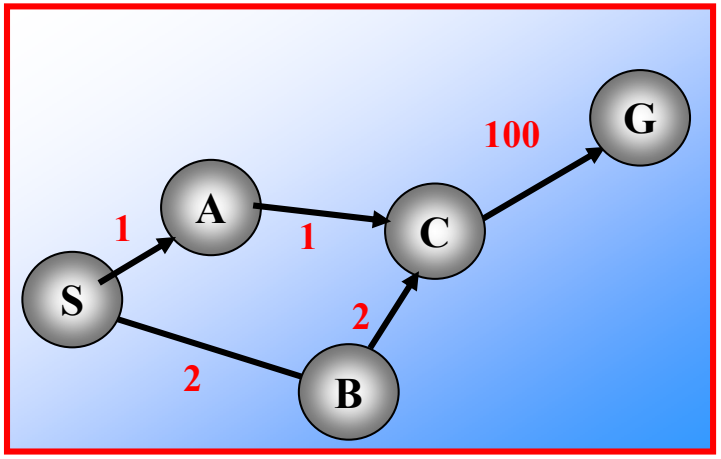
Note that the heuristic is **admissible and consistent**.



?

90-88 ≤ 2 True!

PSEUDOCODE



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

For our convention in class, add the path extensions to the **front** of the Queue.

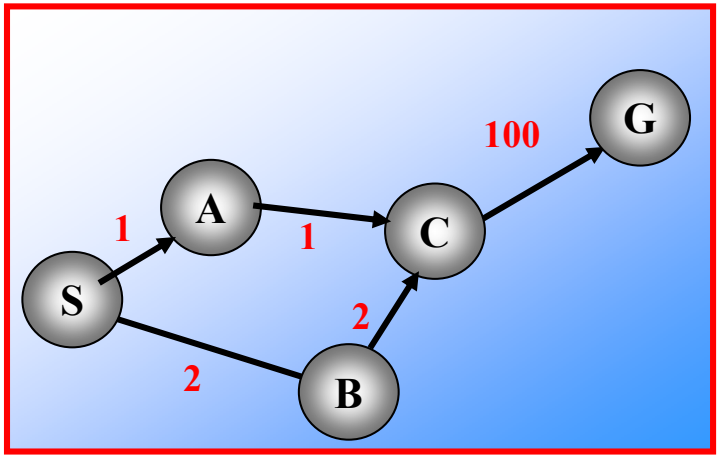
If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)(90 BS)	S
2			
3			
4			
5			

PSEUDOCODE



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

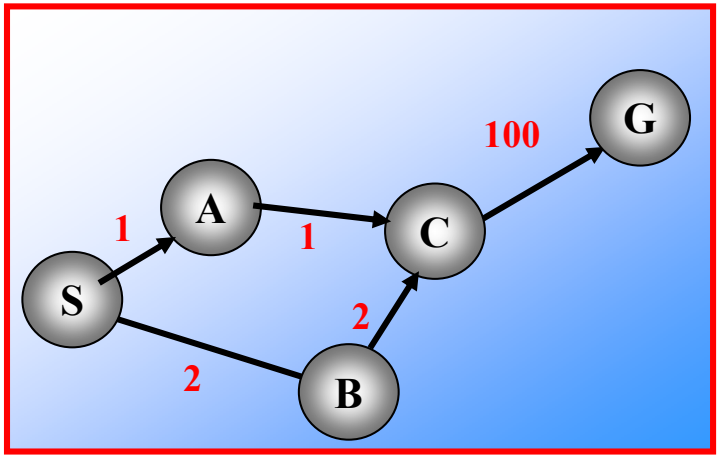
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)(101 AS)	S, B
3			
4			
5			



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

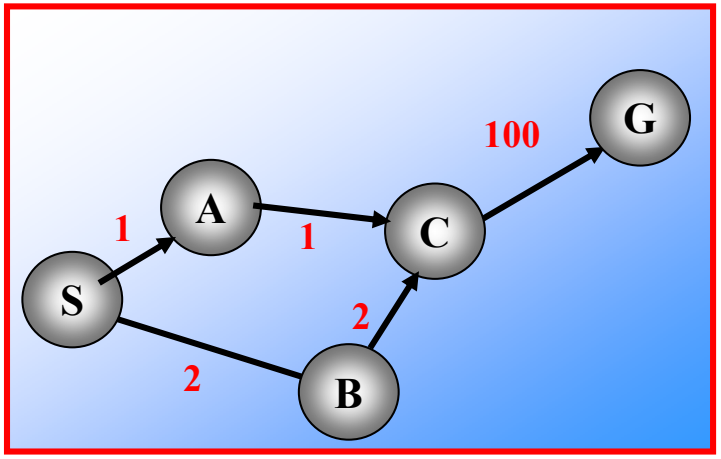
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)~~(101 AS)~~	S, B
3	(101 AS)	(102 CAS)~~(104 CBS)~~	S, B,A
4			
5			



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

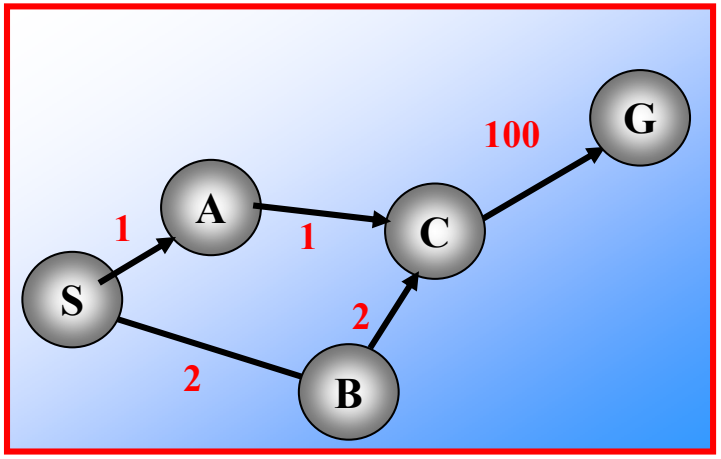
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)~~(101 AS)~~	S, B
3	(101 AS)	~~(102 CAS)~~(~~104 CBS~~)	S, B,A
4	(102 CAS)	(102 GCAS)	S, B, A, C
5			



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

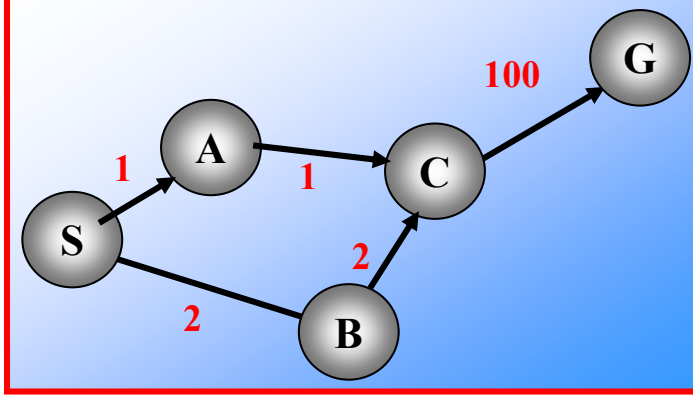
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)~~(101 AS)~~	S, B
3	(101 AS)	~~(102 CAS)~~~~(104 CBS)~~	S, B,A
4	(102 CAS)	~~(102 GCAS)~~	S, B, A, C
5	(102 GCAS)		S, B, A, C, G



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

Dealing with inconsistent heuristic

What can we do if we have an inconsistent heuristic but we still want optimal paths?

Dealing with inconsistent heuristic

What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it **detects** and **corrects** when inconsistency has led us astray.

Dealing with inconsistent heuristic

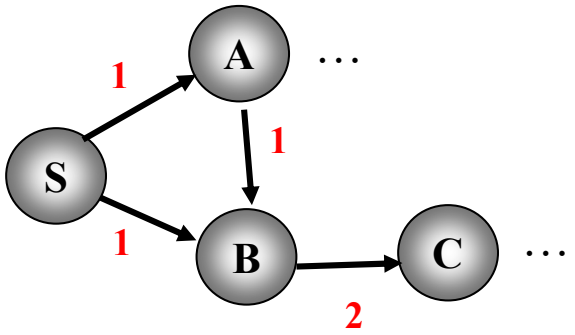
What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it detects and corrects when inconsistency has led us astray.

Assume we are adding $node_1$ to Q and $node_2$ is present in Expanded List with **$node_1$.state = $node_2$.state**.

Example:

We have a node we are considering adding into Q:



(6 CBS) ???

Q
(10 FEDS) …

Search nodes are added in the expanded list

Strict Expanded List
(8 CDAS)

Dealing with inconsistent heuristic

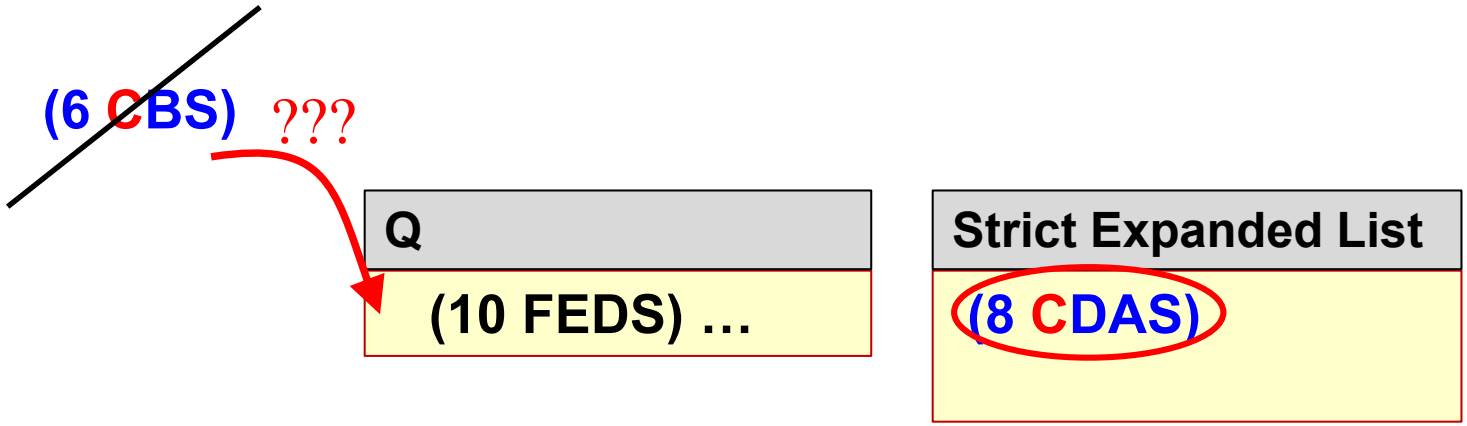
What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it detects and corrects when inconsistency has led us astray.

Assume we are adding $node_1$ to Q and $node_2$ is present in Expanded List with **$node_1$.state = $node_2$.state**.

Strict Expanded List:

- Do NOT add **$node_1$** to Q.



Dealing with inconsistent heuristic

What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it detects and corrects when inconsistency has led us astray.

Assume we are adding $node_1$ to Q and $node_2$ is present in Expanded List with **$node_1$.state = $node_2$.state**.

Strict Expanded List:

- Do NOT add $node_1$ to Q.

Non-Strict Expanded List:

- If (**$node_1$.path_length** < **$node_2$.path_length**), then

  1. Delete $node_2$ from Expanded List
  2. Add $node_1$ to Q.

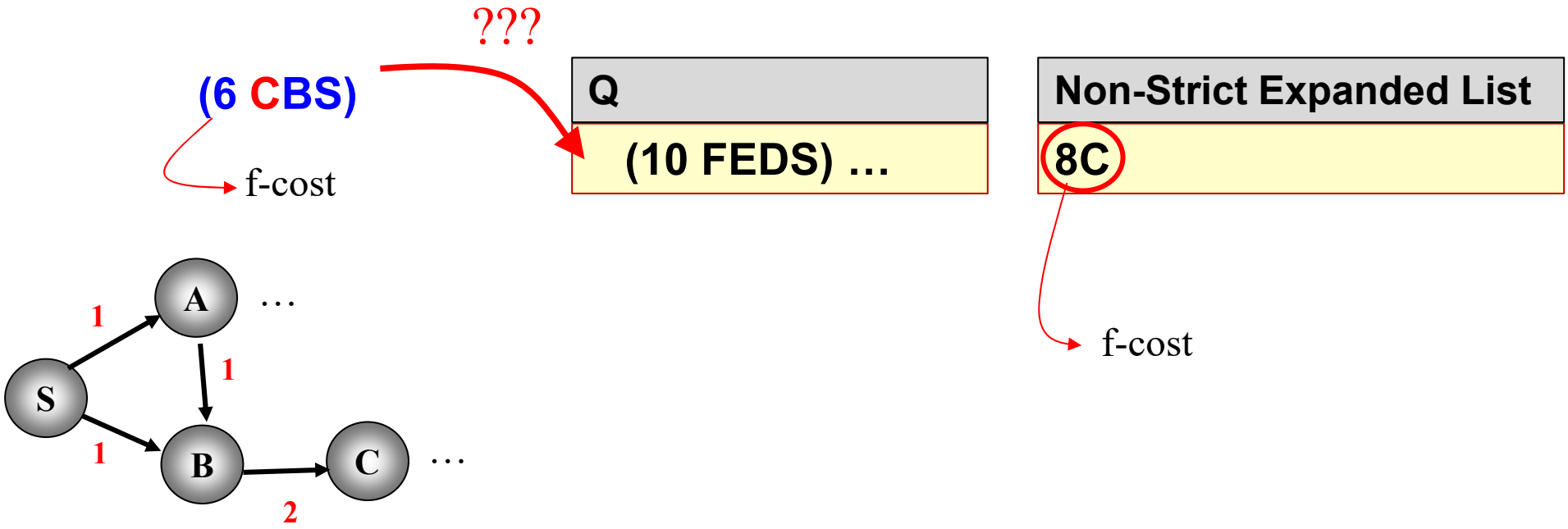
Non-Strict Expanded List

Store the **f-cost** as part of each state recorded



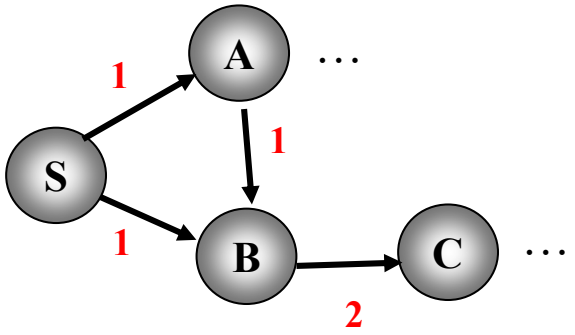
Dealing with inconsistent heuristic

Example: We have a node we are considering adding into Q:



Dealing with inconsistent heuristic

Example: We have a node we are considering adding into Q:



???

(6 CBS)

Q	Non-Strict Expanded List
(10 FEDS) …	8C

6 < 8, and so we should add **(6 CBS)** into the Q, then delete 8C from the Non-Strict Expanded list

Worst Case Complexity

The number of states in the search space may be exponential in some "depth" parameter, e.g. number of actions in a plan, number of moves in a game.

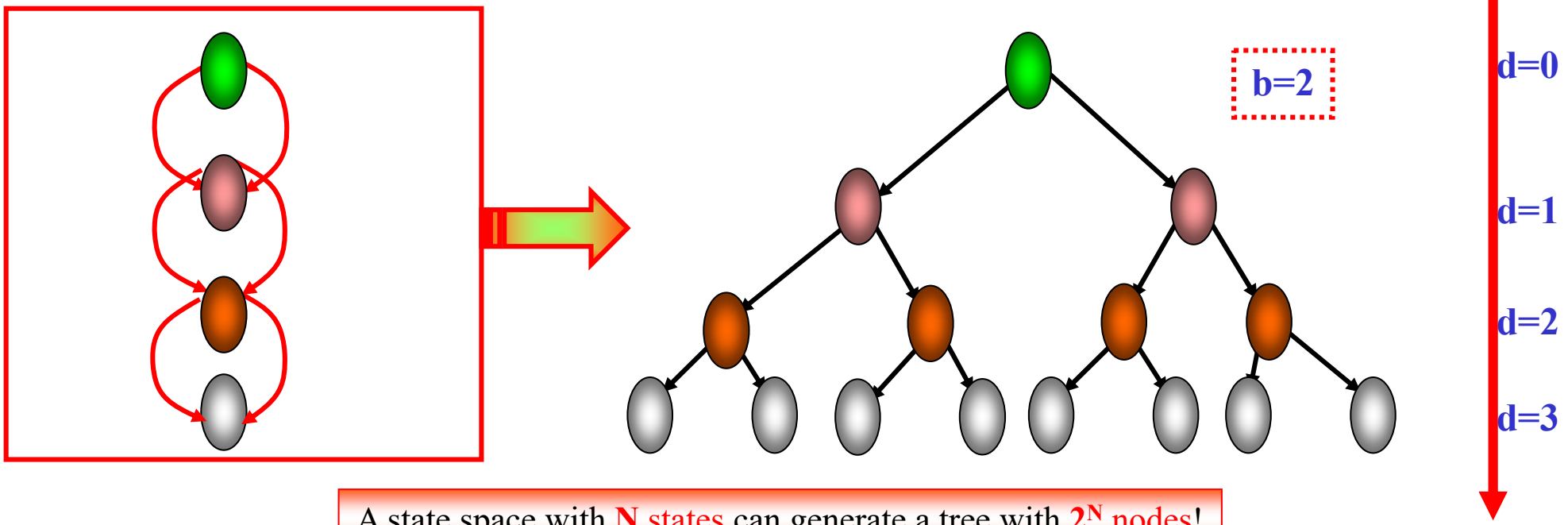
All the searches, with or without Visited or Expanded Lists, **may have to visit** (or expand) **each state** in the worst case.

So, all searches will have **worst case complexities** that are at least **proportional to the number of states** and therefore exponential in the "depth" parameter.

This is the bottom-line irreducible worst case cost of systematic searches.

Worst Case Complexity

- A state space with N states may give rise to a search tree that has a number of nodes that is exponential in N, as in this example.



A state space with N states can generate a tree with 2^N nodes!

- Searches **without** a Visited (Expanded) List may, in the worst case, visit (expand) every node in the search tree.

- Searches with **Strict Visited** (Expanded Lists) will **visit** (expand) **each state only** once

Optimality and Worst Case Complexity

Algorithm	Heuristic	Expanded List	Optimality Guaranteed?	Worst Case & Expansions
UC	None	Strict	Yes	N
A*	Admissible	None	Yes	>N
A*	Consistent	Strict	Yes	N
A*	Admissible	Strict	No	N
A*	Admissible	Non-Strict	Yes	>N